

NATIONAL UNIVERSITY OF MODERN LANGUAGES
ISLAMABAD



PARALLEL AND DIST COMP (LAB)

Lab Report: 01

Submitted to
Mr. Farhad M. Riaz

Submitted By
Aleeha Zahid
(BSAI-136)

Task 1: Hello world with printf

In this task, the C++ program prints a **"Hello world"** message to the console. In addition to displaying the values of the integer variables `id` and `n`, which are 0 and 1, respectively, it outputs the string using the `printf` function. After that, the program ends with a code of 0, telling us that it ran successfully.

```
main.cpp
1  #include <iostream>
2  int main ( ) {
3      int id = 0;
4      int n = 1;
5
6      printf("hello world %d %d", id, n);
7      return 0;
8  }
```

OUTPUT :

```
hello world 0 1

...Program finished with exit code 0
Press ENTER to exit console.
```

Task 2: Parallel threads reporting

In this we used **multithreaded programming** using OpenMP. A parallel region is created by the code and multiple threads are launched to execute the code within this region. After that, each thread sends both its unique ID and the total number of threads generated in this case, eight. Each of the eight threads' messages are displayed in the output in a non-sequential order.

```
main.cpp
1  #include <iostream>
2  #include <stdio.h>
3  #include <omp.h>
4  int main ( ) {
5      #pragma omp parallel
6      {
7          printf("thread %d out of total %d\n", omp_get_thread_num(), omp_get_num_threads());
8      }
9      return 0;
10 }
```

OUTPUT:

```
thread 4 out of total 8
thread 7 out of total 8
thread 1 out of total 8
thread 3 out of total 8
thread 2 out of total 8
thread 5 out of total 8
thread 6 out of total 8
thread 0 out of total 8

...Program finished with exit code 0
Press ENTER to exit console.
```

Task 3: Specifying thread count

This code sets the number of threads to four using the `num_threads`. The program creates four threads, each of which produces a "hello from thread" message before showing its own thread ID. Only threads 0 through 3 were created and executed, as we can see in the output..

```
main.cpp
1  #include <iostream>
2  #include <stdio.h>
3  #include <omp.h>
4  int main () {
5      #pragma omp parallel num_threads(4)
6      {
7          printf("hello from thread %d\n", omp_get_thread_num());
8      }
9
10     return 0;
11 }
```

OUTPUT:

```
hello from thread 1
hello from thread 3
hello from thread 0
hello from thread 2

...Program finished with exit code 0
Press ENTER to exit console.
```

Task 4: Printing from 15 threads

This code sets the number of threads to 15 using the num_threads. The program creates 15 threads, each of which produces a "hello from thread" message before showing its thread ID.

```
main.cpp
1  #include <iostream>
2  #include <stdio.h>
3  #include <omp.h>
4  int main () {
5      #pragma omp parallel num_threads(15)
6      {
7          printf("hello from thread %d\n", omp_get_thread_num());
8      }
9
10 return 0;
11 }
```

OUTPUT:

```
hello from thread 7
hello from thread 8
hello from thread 5
hello from thread 9
hello from thread 10
hello from thread 6
hello from thread 14
hello from thread 13
hello from thread 0
hello from thread 12
hello from thread 2

...Program finished with exit code 0
Press ENTER to exit console. □
```

